

Relatório de Evidências

Laboratório 01 – Parte 2

Disciplina	Banco de Dados, 3º semestre, turno matutino, 2026/2
Docente	Moises Silva de Sousa
Instituição	Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP)
SGBD	MySQL 8.0.46, InnoDB, <code>sql_mode</code> padrão
Repositório	https://github.com/Marcellosiqueira/grupo3_24101166_24101316_24201146

Integrante	Matrícula
Marcello Azevedo Pinheiro Siqueira	24101166
Lucas Basile	24101316
Miguel Matos	24201146

Grupo de três integrantes por exceção autorizada pelo docente, em razão do número de alunos da turma.

1 Ambiente e reprodução

Todas as evidências deste relatório foram produzidas no MySQL 8.0.46, com o `sql_mode` padrão da instalação, incluindo `ONLY_FULL_GROUP_BY` e `STRICT_TRANS_TABLES`. Todas as tabelas usam o motor InnoDB e o conjunto de caracteres `utf8mb4`.

Os arquivos que reproduzem o que está aqui, todos no diretório `sql/` do repositório:

Laboratorio01_Grupo3.sql

Recria o banco `locadora` do zero, cria as sete tabelas, insere a carga completa e executa as seis consultas obrigatórias da Parte 1.

script_parte2.sql

Entregável da Parte 2. Cria as três views, o índice novo, a tabela de auditoria, os três triggers, a procedure e a function. É re-executável: pode rodar quantas vezes for necessário sem erro e sem efeito colateral.

parte3_concorrencia.sql

Roteiro de execução da Parte 3. Não roda de uma vez: cada comando indica a sessão responsável e o passo, e é executado alternando entre duas conexões simultâneas. Define também a `sp_abrir_locacao_seguro`, discutida na Seção 7.5.

evidencias_explain.sql

Não faz parte do entregável executável. Derruba os três índices, roda `EXPLAIN` nas três consultas, recria os índices e roda `EXPLAIN` de novo, apenas para produzir as evidências da Seção 3.

Ordem de execução:

```
mysql -uroot -p          < sql/Laboratorio01_Grupo3.sql
mysql -uroot -p locadora < sql/script_parte2.sql
mysql -uroot -p locadora < sql/evidencias_explain.sql  -- apenas para gerar as evidencias
```

1.1 Objetos criados

```

-----
SELECT TABLE_NAME AS view_name, IS_UPDATABLE FROM information_schema.VIEWS WHERE TABLE_SCHEMA = 'locadora' ORDER BY TABLE_NAME
-----

+-----+-----+
| view_name | IS_UPDATABLE |
+-----+-----+
| vw_faturamento_mensal | NO |
| vw_faturamento_veiculo | NO |
| vw_locacoes_ativas | YES |
| vw_locacoes_ativas_join | YES |
| vw_locacoes_mes | YES |
| vw_veiculos_disponiveis | YES |
| vw_veiculos_disponiveis_simples | YES |
+-----+-----+

SELECT TRIGGER_NAME, EVENT_MANIPULATION, ACTION_TIMING FROM information_schema.TRIGGERS WHERE TRIGGER_SCHEMA = 'locadora' ORDER BY TRIGGER_NAME
-----

+-----+-----+-----+
| TRIGGER_NAME | EVENT_MANIPULATION | ACTION_TIMING |
+-----+-----+-----+
| trg_locacao_ai_status | INSERT | AFTER |
| trg_locacao_au_auditoria | UPDATE | AFTER |
| trg_locacao_au_status | UPDATE | AFTER |
+-----+-----+-----+

SELECT ROUTINE_NAME, ROUTINE_TYPE FROM information_schema.ROUTINES WHERE ROUTINE_SCHEMA = 'locadora' ORDER BY ROUTINE_NAME
-----

+-----+-----+
| ROUTINE_NAME | ROUTINE_TYPE |
+-----+-----+
| fn_calcula_multa | FUNCTION |
| sp_abrir_locacao | PROCEDURE |
| sp_refresh_mv_faturamento_veiculo | PROCEDURE |
+-----+-----+

```

As views `vw_faturamento_veiculo`, `vw_locacoes_ativas_join`, `vw_locacoes_mes` e `vw_veiculos_disponiveis_simples`, e a procedure `sp_refresh_mv_faturamento_veiculo`, são da atividade de Views e Índices da Aula 14, executada no mesmo banco. Estão listadas porque o inventário é do esquema inteiro. A Seção 8.2 explica o motivo dos dois sufixos.

2 Parte 1.1 – Views

As três views pedidas pelo roteiro foram criadas com `CREATE OR REPLACE VIEW`, o que torna o script re-executável.

vw_locacoes_ativas

Cliente, veículo, filiais e datas de retirada e de devolução prevista das locações que ainda não têm data real de devolução, ou seja, `data_real_devolucao IS NULL`.

vw_veiculos_disponiveis

Veículos com `status = 'DISPONIVEL'`, com a categoria e o valor da diária vindos de `categoria`, e a filial atual.

vw_faturamento_mensal

Soma de `valor_total` das locações finalizadas, agrupada por filial de retirada e por ano e mês da retirada.

Resultado das três:

```

-----
SELECT id_locacao, cliente, placa, veiculo, data_retirada, data_prevista_devolucao FROM vw_locacoes_ativas
-----

+-----+-----+-----+-----+-----+-----+
| id_locacao | cliente | placa | veiculo | data_retirada | data_prevista_devolucao |
+-----+-----+-----+-----+-----+-----+
| 11 | Marcos Vinicius Andrade | JKL5C21 | Toyota Corolla Cross | 2026-09-12 09:00:00 | 2026-09-19 09:00:00 |
| 12 | Juliana Rezende Castro | ICV9F03 | Jeep Compass | 2026-09-14 10:00:00 | 2026-09-21 10:00:00 |
| 13 | Marcos Vinicius Andrade | QAC2D34 | Chevrolet Onix | 2026-09-10 09:00:00 | 2026-09-17 09:00:00 |
| 14 | Rafael Bittencourt Souza | QAF5G67 | Fiat Cronos | 2026-09-11 10:00:00 | 2026-09-18 10:00:00 |
| 15 | Camila Duarte Ferreira | QAK9L01 | Volkswagen T-Cross | 2026-09-13 08:30:00 | 2026-09-20 08:30:00 |
| 16 | Thiago Moreira Alencar | QAQ4R56 | Audi A4 | 2026-09-15 14:00:00 | 2026-09-18 14:00:00 |

```

```

| 17 | Juliana Rezende Castro | QAV9W01 | Ford Transit | 2026-09-15 07:00:00 | 2026-09-22 07:00:00 |
+-----+-----+-----+-----+-----+-----+
SELECT placa, marca, modelo, categoria, valor_diaria, filial_atual FROM vw_veiculos_disponiveis LIMIT 6
-----
+-----+-----+-----+-----+-----+-----+
| placa | marca | modelo | categoria | valor_diaria | filial_atual |
+-----+-----+-----+-----+-----+-----+
| RTB7D45 | Fiat | Argo | ECONOMICO | 129.90 | Filial Asa Sul |
| BND4G12 | Chevrolet | Onix Plus | INTERMEDIARIO | 189.90 | Filial Taguatinga |
| LWS8H27 | BMW | 320i | LUXO | 549.90 | Filial Aeroporto |
| PDF3J56 | Fiat | Fiorino | UTILITARIO | 219.90 | Filial Aguas Claras |
| GHT6K91 | Renault | Kwid | ECONOMICO | 129.90 | Filial Aguas Claras |
| QAB1C23 | Volkswagen | Polo Track | ECONOMICO | 129.90 | Filial Taguatinga |
+-----+-----+-----+-----+-----+-----+
SELECT filial_retirada, ano_mes, total_locacoes, faturamento_total FROM vw_faturamento_mensal
-----
+-----+-----+-----+-----+
| filial_retirada | ano_mes | total_locacoes | faturamento_total |
+-----+-----+-----+-----+
| Filial Asa Sul | 2026-09 | 1 | 1039.20 |
| Filial Taguatinga | 2026-09 | 1 | 759.60 |
| Filial Aguas Claras | 2026-08 | 2 | 2058.90 |
| Filial Asa Norte | 2026-08 | 1 | 649.50 |
| Filial Aeroporto | 2026-07 | 1 | 3299.40 |
| Filial Asa Norte | 2026-07 | 1 | 2029.30 |
| Filial Taguatinga | 2026-07 | 1 | 569.70 |
| Filial Asa Sul | 2026-06 | 2 | 2968.70 |
+-----+-----+-----+-----+

```

Uma observação sobre a `vw_faturamento_mensal`: ela soma apenas locações com `status = 'FINALIZADA'`. No nosso modelo, `valor_total` só é preenchido na devolução, então locação em aberto tem `valor_total` nulo e não teria o que somar. Incluir locações canceladas inflaria o faturamento com contratos que não aconteceram.

3 Parte 1.2 – EXPLAIN antes e depois dos índices

As três consultas analisadas, exatamente como estão no `evidencias_explain.sql`:

```

-- Consulta A: busca de cliente por CPF
SELECT id_cliente, nome, cpf, telefone
  FROM cliente
 WHERE cpf = '07894561203';

-- Consulta B: locacoes filtradas por intervalo de data de retirada
SELECT id_locacao, id_cliente, id_veiculo, data_retirada, valor_total
  FROM locacao
 WHERE data_retirada BETWEEN '2026-08-01' AND '2026-09-30';

-- Consulta C: veiculos filtrados por categoria
SELECT id_veiculo, placa, marca, modelo, status
  FROM veiculo
 WHERE id_categoria = 3;

```

Os índices usados em cada uma:

Consulta	Índice	Origem
A	<code>uq_cliente_cpf</code>	Restrição UNIQUE sobre <code>cliente.cpf</code> , já existente desde a Etapa 4
B	<code>idx_locacao_data_retirada</code>	Criado nesta Parte 2, sobre <code>locacao (data_retirada)</code>
C	<code>idx_veiculo_categoria</code>	Índice composto sobre <code>veiculo (id_categoria, status)</code> , já existente desde a Etapa 4

Como dois dos três índices já existiam, o cenário “antes” exigiu derrubá-los primeiro. Um deles não sai sozinho, o que virou uma conclusão do exercício e está na Seção 8.1.

3.1 Antes dos índices

```

+-----+
| cenário |
+-----+
| CONSULTA A - cliente por CPF - ANTES |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | cliente | NULL | ALL | NULL | NULL | NULL | NULL | 6 | 16.67 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| cenário |
+-----+
| CONSULTA B - locacoes por intervalo de data - ANTES |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | locacao | NULL | ALL | NULL | NULL | NULL | NULL | 17 | 11.11 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| cenário |
+-----+
| CONSULTA C - veiculos por categoria - ANTES |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | veiculo | NULL | ALL | NULL | NULL | NULL | NULL | 30 | 10.00 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

3.2 Depois dos índices

```

+-----+
| cenário |
+-----+
| CONSULTA A - cliente por CPF - DEPOIS |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | cliente | NULL | const | uq_cliente_cpf | uq_cliente_cpf | 44 | const | 1 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| cenário |
+-----+
| CONSULTA B - locacoes por intervalo de data - DEPOIS |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | locacao | NULL | ALL | idx_locacao_data_retirada | NULL | NULL | NULL | 17 | 70.59 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| cenário |
+-----+
| CONSULTA C - veiculos por categoria - DEPOIS |
+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | veiculo | NULL | ref | idx_veiculo_categoria | idx_veiculo_categoria | 4 | const | 6 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

3.3 Leitura dos planos

Consulta	Antes	Depois	Linhas	filtered
A	ALL, key nulo	const, uq_cliente_cpf	6 para 1	16,67% para 100%
B	ALL, key nulo	ALL, key nulo	17 para 17	11,11% para 70,59%
C	ALL, key nulo	ref, idx_veiculo_categoria	30 para 6	10% para 100%

Consulta A. O ganho é o maior dos três. O acesso passou de varredura completa da tabela para *const*, que é o tipo mais barato do MySQL: o otimizador sabe que uma comparação de igualdade contra um índice único devolve no máximo uma linha, então a linha é lida uma vez e tratada como constante pelo resto do plano.

Consulta C. O acesso passou para `ref`, que é a busca por prefixo de índice não único. O índice é composto por (`id_categoria`, `status`) e a consulta filtra só por `id_categoria`, o primeiro campo, então o prefixo à esquerda serve. As 30 linhas examinadas caíram para 6, que é exatamente a quantidade de veículos da categoria 3, e o `filtered` foi a 100%, indicando que toda linha lida pelo índice interessa.

Consulta B. O índice foi criado e o otimizador não o usou. Ele aparece em `possible_keys`, mas `key` continua nulo e o tipo continua `ALL`. Isso não é falha do índice nem erro de criação: o intervalo pedido pelo roteiro, de 01/08/2026 a 30/09/2026, cobre 12 das 17 locações da carga. A evidência de que o otimizador considerou o índice e o descartou por cálculo, e não por ignorá-lo, é a mudança do `filtered`: antes de existir o índice, o valor era 11,11%, que é o palpite padrão do MySQL para um `BETWEEN` sem estatística; depois, passou a 70,59%, que é a fração real, ou seja, 12 dividido por 17. Com 70% das linhas dentro do intervalo, percorrer o índice e depois buscar cada linha correspondente na chave primária custa mais que ler as 17 linhas em sequência.

A conclusão prática é que a utilidade de um índice depende da seletividade do predicado e do tamanho da tabela, não da existência do índice. O mesmo `idx_locacao_data_retirada` passaria a ser usado num intervalo estreito, por exemplo um único dia, ou numa tabela com volume de produção. Ele foi mantido no entregável porque o critério de escolha do índice é a consulta que a aplicação faz com frequência, e não o resultado de um caso de teste com 17 linhas.

4 Pergunta de reflexão

Em que situações a presença de múltiplos índices pode prejudicar o desempenho do banco de dados, por exemplo em tabelas com alta taxa de `INSERT`, `UPDATE` ou `DELETE`?

Um índice é uma estrutura redundante mantida pelo SGBD em troca de leitura mais rápida. Quem paga essa troca é a escrita. Os pontos em que a conta fica desfavorável são os seguintes.

Custo por operação de escrita. No InnoDB, cada índice secundário é uma árvore B+ própria. Um `INSERT` numa tabela com oito índices secundários não escreve uma vez: escreve a linha no índice agrupado e mais oito entradas, cada uma com sua localização na árvore, possível divisão de página e seu registro no log de redo. O `DELETE` paga o mesmo, e o `UPDATE` paga por cada índice que contenha alguma das colunas alteradas. Numa tabela de alto volume de escrita, como seria a `locacao` de uma locadora real, esse custo multiplica diretamente o tempo de cada transação.

Divisão de páginas e fragmentação. Índice sobre coluna cujos valores chegam em ordem crescente, como uma chave substituta autoincremento, insere sempre no fim da árvore e é barato. Índice sobre coluna de valor disperso, como CPF, placa ou UUID, insere no meio, o que provoca divisão de páginas, deixa páginas parcialmente ocupadas e faz o índice ocupar mais espaço do que o conteúdo exigiria. O efeito aparece depois, como leitura mais lenta por menos densidade de dados por página.

Competição por memória. Os índices disputam o *buffer pool* com os dados. Um índice pouco consultado continua tendo suas páginas carregadas a cada escrita e ocupa espaço que poderia estar guardando páginas quentes. O sintoma é aumento de leitura em disco sem mudança na carga de consulta.

Bloqueio e concorrência. No InnoDB o bloqueio de linha é aplicado sobre registros de índice, não sobre a linha física. Quanto mais índices cobrem as colunas alteradas por uma transação, mais registros de índice ela bloqueia, e maior a superfície de contenção com outras transações. No nível de isolamento padrão do MySQL, `REPEATABLE READ`, somam-se ainda os *gap locks*, que travam intervalos do índice e não apenas as linhas existentes. O resultado é mais espera e mais chance de *deadlock*, que é justamente o fenômeno do Experimento 3.2.

Custo de planejamento e risco de plano pior. O otimizador avalia os índices candidatos antes de escolher o plano. Com muitos índices sobrepostos, o tempo de planejamento cresce e aumenta a chance de escolha ruim, porque as estatísticas do InnoDB são amostradas e podem ficar desatualizadas entre execuções de `ANALYZE TABLE`.

Índices redundantes. Um índice sobre (a) é inútil quando já existe um sobre (a, b), porque o prefixo à esquerda do composto já atende. O índice redundante paga todos os custos acima e não traz nenhum benefício. No nosso modelo, um índice sobre `veiculo` (`id_categoria`) seria exatamente esse caso, já que `idx_veiculo_categoria` é (`id_categoria`, `status`).

Índice que o otimizador nunca escolhe. É o caso medido na nossa Consulta B. O índice existe, é mantido em toda escrita na tabela `locacao` e o plano de execução não o utiliza. Custo integral, benefício zero, para aquele padrão de consulta.

O critério que decorre disso é indexar as colunas que aparecem em filtro, junção e ordenação das consultas frequentes, medir com `EXPLAIN` depois de criar, e remover o índice que o otimizador nunca escolhe e que nenhuma restrição de integridade exige.

5 Parte 2.1 – Triggers

Três triggers foram criados sobre a tabela `locacao`.

5.1 Status do veículo na abertura

`trg_locacao_ai_status`, AFTER INSERT: muda o status do veículo para `LOCADO`. O gatilho só dispara quando a locação entra realmente em aberto, isto é, quando `NEW.data_real_devolucao` IS NULL e `NEW.status = 'ABERTA'`. Sem essa guarda, a carga histórica do Laboratório 01, que insere locações já finalizadas, marcaria como locados veículos que estão disponíveis.

5.2 Status do veículo na devolução

`trg_locacao_au_status`, AFTER UPDATE: quando a data real de devolução é preenchida, devolve o veículo para `DISPONIVEL` e atualiza `veiculo.km_atual` com a quilometragem registrada na devolução.

5.3 Auditoria

`trg_locacao_au_auditoria`, AFTER UPDATE: grava uma linha em `log_locacao` sempre que `valor_total` ou `status` mudam. A comparação usa o operador `<=>`, seguro para nulo, porque `valor_total` é nulo enquanto a locação está aberta e `OLD.valor_total <> NEW.valor_total` não detectaria a transição de nulo para valor.

A tabela `log_locacao` tem a estrutura mínima pedida pelo roteiro: `id_log`, `id_locacao`, `valor_antigo`, `valor_novo`, `status_antigo`, `status_novo`, `usuario` e `data_alteracao`. O campo `usuario` é preenchido com `CURRENT_USER()` e `data_alteracao` com `CURRENT_TIMESTAMP`.

6 Parte 2.2 – Procedure e function

6.1 `sp_abrir_locacao`

Recebe `id_cliente`, `id_veiculo`, `id_filial` e `dias_locacao`. Valida a disponibilidade do veículo, busca a diária na categoria, calcula o valor previsto e insere a locação. Devolve por parâmetro de saída o identificador criado e o valor previsto.

Evidência do caminho de sucesso, que também demonstra o trigger `trg_locacao_ai_status`:

```
-----
SELECT id_veiculo, placa, status FROM veiculo WHERE id_veiculo = 2
-----
```

```

+-----+-----+-----+
| id_veiculo | placa | status |
+-----+-----+-----+
|          2 | RTB7D45 | DISPONIVEL |
+-----+-----+-----+

CALL sp_abrir_locacao(2, 2, 1, 5, @id_locacao, @valor)

-----

SELECT @id_locacao AS id_locacao_criada, @valor AS valor_previsto
-----

+-----+-----+-----+
| id_locacao_criada | valor_previsto |
+-----+-----+-----+
|          18 |          649.50 |
+-----+-----+-----+

SELECT id_locacao, id_cliente, id_veiculo, status, valor_total FROM locacao WHERE id_locacao = @id_locacao
-----

+-----+-----+-----+-----+-----+
| id_locacao | id_cliente | id_veiculo | status | valor_total |
+-----+-----+-----+-----+-----+
|          18 |          2 |          2 | ABERTA |          NULL |
+-----+-----+-----+-----+-----+

SELECT id_veiculo, placa, status FROM veiculo WHERE id_veiculo = 2
-----

+-----+-----+-----+
| id_veiculo | placa | status |
+-----+-----+-----+
|          2 | RTB7D45 | LOCADO |
+-----+-----+-----+

```

O veículo 2 estava **DISPONIVEL** antes da chamada e ficou **LOCADO** depois, sem que nenhum comando tenha alterado a tabela **veiculo**: quem fez isso foi o trigger.

A locação nasce com **valor_total** nulo, e não com o valor previsto. Essa é uma adaptação deliberada ao nosso modelo: o dicionário de dados entregue na Parte 1 define **valor_total** como o valor efetivamente cobrado, apurado na devolução, porque a devolução pode ser antecipada ou atrasada. Gravar o previsto nesse campo quebraria a semântica já documentada e faria a **vw_faturamento_mensal** somar receita de contratos ainda em aberto. O valor previsto é devolvido pelo parâmetro de saída **p_valor_previsto**, que é onde a aplicação o consome.

Evidência da validação, com o mesmo veículo agora locado:

```

-----
CALL sp_abrir_locacao(3, 2, 1, 5, @x, @y)
-----

ERROR 1644 (45000) at line 1: sp_abrir_locacao: veiculo nao esta disponivel

```

O erro é emitido por **SIGNAL SQLSTATE '45000'**, que é a forma padrão de lançar exceção definida pelo usuário no MySQL, e chega à aplicação como **ERROR 1644**.

6.2 fn_calcula_multa

Recebe **id_locacao** e **taxa_diaria_multa**, calcula a diferença em dias entre a data prevista e a data real de devolução com **DATEDIFF** e devolve o valor da multa. Sem atraso, ou com a locação ainda em aberto, devolve zero. É declarada **READS SQL DATA** porque consulta a tabela **locacao** e não a modifica.

Evidência da devolução com três dias de atraso, que demonstra de uma vez os dois triggers de **UPDATE** e a function:

```

-----
UPDATE locacao
  SET data_real_devolucao = DATE_ADD(data_prevista_devolucao, INTERVAL 3 DAY),
      status              = 'FINALIZADA',
      valor_total         = 649.50,
      km_devolucao        = km_retirada + 400
  WHERE id_locacao = 18
-----

SELECT id_veiculo, placa, status, km_atual FROM veiculo WHERE id_veiculo = 2
-----

+-----+-----+-----+-----+
| id_veiculo | placa | status | km_atual |
+-----+-----+-----+-----+

```

```

+-----+-----+-----+-----+
|      2 | RTB7D45 | DISPONIVEL | 13800 |
+-----+-----+-----+-----+
SELECT id_log, id_locacao, valor_antigo, valor_novo, status_antigo, status_novo, usuario FROM log_locacao
-----

+-----+-----+-----+-----+-----+-----+-----+
| id_log | id_locacao | valor_antigo | valor_novo | status_antigo | status_novo | usuario |
+-----+-----+-----+-----+-----+-----+-----+
|      1 |          18 |          NULL |       649.50 |          ABERTA |          FINALIZADA | root@localhost |
+-----+-----+-----+-----+-----+-----+-----+

SELECT fn_calcula_multa(18, 50.00) AS multa_com_3_dias_de_atraso,
       fn_calcula_multa(1, 50.00) AS multa_devolvida_no_prazo,
       fn_calcula_multa(11, 50.00) AS multa_locacao_ainda_aberta
-----

+-----+-----+-----+
| multa_com_3_dias_de_atraso | multa_devolvida_no_prazo | multa_locacao_ainda_aberta |
+-----+-----+-----+
|          150.00 |          0.00 |          0.00 |
+-----+-----+-----+

```

O UPDATE tocou apenas a tabela locacao. O veículo voltou para DISPONIVEL e teve o km_atual atualizado por trg_locacao_au_status; a linha em log_locacao foi gravada por trg_locacao_au_auditoria, registrando a transição de valor_total nulo para 649,50 e de ABERTA para FINALIZADA. A multa de 150,00 corresponde a três dias vezes a taxa de 50,00.

7 Parte 3 – Concorrência e transações

Os experimentos foram executados em duas conexões simultâneas e independentes ao mesmo banco locadora, identificadas abaixo como Sessão A e Sessão B, com os comandos adaptados aos nomes de tabelas e colunas do nosso modelo físico. As Seções 7.1 a 7.3 cobrem os três experimentos do roteiro. As Seções 7.4 e 7.5 são um aprofundamento do grupo sobre os mesmos mecanismos, aplicado às rotinas que a Parte 2 entregou. Todo o roteiro de execução está no arquivo sql/parte3_concorrencia.sql.

7.1 Experimento 3.1 – Bloqueio explícito com FOR UPDATE

```

-- Sessao A
BEGIN;
SELECT * FROM veiculo WHERE id_veiculo = 1 FOR UPDATE;  -- nao commitar ainda

-- Sessao B
BEGIN;
SELECT * FROM veiculo WHERE id_veiculo = 1 FOR UPDATE;  -- fica bloqueada

-- Sessao A
COMMIT;  -- a Sessao B e liberada

-- Sessao B
COMMIT;

```

```

Sessao A> BEGIN;
Sessao A> SELECT * FROM veiculo WHERE id_veiculo = 1 FOR UPDATE;
          (1, 'JKL5C21', 'Corolla Cross', 'LOCADO')
          (transacao aberta, sem COMMIT)

Sessao B> BEGIN;
Sessao B> SELECT * FROM veiculo WHERE id_veiculo = 1 FOR UPDATE;
          (bloqueada ha 3 s, aguardando o lock da Sessao A)

Sessao A> COMMIT;
Sessao B> (liberada apos 3.04 s de espera)
          (1, 'JKL5C21', 'Corolla Cross', 'LOCADO')
Sessao B> COMMIT;

```

A Sessão A foi mantida com a transação aberta por três segundos. A Sessão B ficou parada durante todo esse intervalo e recebeu a linha 3,04 segundos depois de ter pedido, ou seja, imediatamente após o COMMIT da Sessão A. O FOR UPDATE coloca um bloqueio exclusivo sobre o registro de índice da linha, e não apenas uma leitura consistente, então a segunda transação que pede a mesma linha no mesmo modo não recebe uma versão antiga: ela espera. É esse o comportamento que impede duas reservas simultâneas do mesmo veículo.

7.2 Experimento 3.2 – Indução de deadlock

```
-- 1. Sessao A
BEGIN;
-- 2. Sessao A
UPDATE cliente SET nome = 'Teste A' WHERE id_cliente = 1;
-- 3. Sessao B
BEGIN;
-- 4. Sessao B
UPDATE veiculo SET modelo = 'Teste B' WHERE id_veiculo = 1;
-- 5. Sessao A (fica aguardando a Sessao B)
UPDATE veiculo SET modelo = 'Teste A' WHERE id_veiculo = 1;
-- 6. Sessao B (provoca o deadlock)
UPDATE cliente SET nome = 'Teste B' WHERE id_cliente = 1;
```

```
Passo 1 Sessao A> BEGIN;
Passo 2 Sessao A> UPDATE cliente SET nome = 'Teste A' WHERE id_cliente = 1;
          Query OK, 1 row affected

Passo 3 Sessao B> BEGIN;
Passo 4 Sessao B> UPDATE veiculo SET modelo = 'Teste B' WHERE id_veiculo = 1;
          Query OK, 1 row affected

Passo 5 Sessao A> UPDATE veiculo SET modelo = 'Teste A' WHERE id_veiculo = 1;
          (aguardando o lock da Sessao B)

Passo 6 Sessao B> UPDATE cliente SET nome = 'Teste B' WHERE id_cliente = 1;
          Sessao A: Query OK, 1 row affected
          Sessao B: ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction
```

Mensagem oficial do SGBD, obtida com `SHOW ENGINE INNODB STATUS`. As linhas de despejo hexadecimal dos registros foram omitidas por não acrescentarem informação ao diagnóstico:

```
LATEST DETECTED DEADLOCK
-----
2026-09-28 14:12:33 140208591955648
*** (1) TRANSACTION:
TRANSACTION 12423, ACTIVE 2 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 4 lock struct(s), heap size 1128, 2 row lock(s), undo log entries 1
MySQL thread id 89, OS thread handle 140208541599424, query id 1169 localhost root updating
UPDATE veiculo SET modelo='Teste A' WHERE id_veiculo=1

*** (1) HOLDS THE LOCK(S):
RECORD LOCKS space id 287 page no 4 n bits 80 index PRIMARY of table 'locadora'.'cliente' trx id 12423 lock_mode X locks rec but not gap

*** (1) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 290 page no 4 n bits 104 index PRIMARY of table 'locadora'.'veiculo' trx id 12423 lock_mode X locks rec but not gap waiting

*** (2) TRANSACTION:
TRANSACTION 12424, ACTIVE 2 sec starting index read
mysql tables in use 1, locked 1
LOCK WAIT 4 lock struct(s), heap size 1128, 2 row lock(s), undo log entries 1
MySQL thread id 90, OS thread handle 140208874923712, query id 1170 localhost root updating
UPDATE cliente SET nome='Teste B' WHERE id_cliente=1

*** (2) HOLDS THE LOCK(S):
RECORD LOCKS space id 290 page no 4 n bits 104 index PRIMARY of table 'locadora'.'veiculo' trx id 12424 lock_mode X locks rec but not gap

*** (2) WAITING FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 287 page no 4 n bits 80 index PRIMARY of table 'locadora'.'cliente' trx id 12424 lock_mode X locks rec but not gap waiting

*** WE ROLL BACK TRANSACTION (2)
```

O relatório do InnoDB descreve o ciclo inteiro. A transação 12423, que é a Sessão A, segura um bloqueio exclusivo sobre a chave primária de **cliente** e espera pela chave primária de **veiculo**. A transação 12424, que é a Sessão B, segura exatamente o oposto: tem **veiculo** e espera **cliente**. Nenhuma das duas pode avançar sem que a outra libere, e nenhuma vai liberar antes de avançar. O InnoDB detecta o ciclo e escolhe uma vítima, informada na última linha, `WE ROLL BACK TRANSACTION (2)`: a Sessão B foi desfeita e recebeu `ERROR 1213 (40001)`, enquanto a Sessão A seguiu normalmente. O critério de escolha é o custo de desfazer, medido em número de linhas modificadas.

Prevenção por ordem padronizada de acesso. O deadlock acontece porque as duas transações pegam os mesmos dois recursos em ordens opostas: a Sessão A trava **cliente** e depois pede **veiculo**; a Sessão B trava **veiculo** e depois pede **cliente**. Se as duas acessassem sempre na mesma ordem, por exemplo **cliente** antes de **veiculo**, a segunda transação ficaria bloqueada logo no primeiro recurso e apenas esperaria a primeira terminar, o que é espera comum e resolve sozinha. Não existe ciclo quando existe ordem total.

A regra que se tira disso tem três partes. Primeira, fixar uma ordem canônica de acesso às tabelas dentro de cada rotina que abre transação, e concentrar essa lógica em procedures como a `sp_abrir_locacao`, em vez de deixar cada ponto da aplicação escolher a sua ordem. Segunda, manter as transações curtas, porque o bloqueio dura enquanto a transação durar e a janela para o ciclo se formar é proporcional a esse tempo. Terceira, tratar **ERROR 1213** na aplicação como erro recuperável, refazendo a transação, já que o próprio SGBD sugere isso na mensagem: deadlock é condição esperada em sistema concorrente, não defeito.

7.3 Experimento 3.3 – Níveis de isolamento e MVCC

Uma ressalva de ambiente antes do resultado: o roteiro trata **READ COMMITTED** como nível padrão, o que vale para o PostgreSQL e para o SQL Server, mas não para o MySQL, cujo padrão no InnoDB é **REPEATABLE READ**. Por isso a Sessão B declarou o nível explicitamente em cada rodada, e o experimento foi feito duas vezes, uma em cada nível, o que também atende ao passo 5 do roteiro.

```
-- Sessao B, antes de abrir a transacao
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED; -- e depois REPEATABLE READ
SELECT @@transaction_isolation;
```

```
=== Sessao B com nivel READ-COMMITTED ===

Sessao A> BEGIN;
Sessao A> UPDATE cliente SET nome = 'Alterado pela Sessao A' WHERE id_cliente = 1; -- sem COMMIT

Sessao B> BEGIN;
Sessao B> SELECT nome FROM cliente WHERE id_cliente = 1;
           'Marcos Vinicius Andrade'      <- leitura 1, antes do COMMIT de A

Sessao A> COMMIT;

Sessao B> SELECT nome FROM cliente WHERE id_cliente = 1; -- mesma transacao de B
           'Alterado pela Sessao A'      <- leitura 2, depois do COMMIT de A
Sessao B> COMMIT;
Sessao B> SELECT nome FROM cliente WHERE id_cliente = 1; -- nova transacao de B
           'Alterado pela Sessao A'      <- leitura 3

=== Sessao B com nivel REPEATABLE-READ ===

Sessao A> BEGIN;
Sessao A> UPDATE cliente SET nome = 'Alterado pela Sessao A' WHERE id_cliente = 1; -- sem COMMIT

Sessao B> BEGIN;
Sessao B> SELECT nome FROM cliente WHERE id_cliente = 1;
           'Marcos Vinicius Andrade'      <- leitura 1, antes do COMMIT de A

Sessao A> COMMIT;

Sessao B> SELECT nome FROM cliente WHERE id_cliente = 1; -- mesma transacao de B
           'Marcos Vinicius Andrade'      <- leitura 2, depois do COMMIT de A
Sessao B> COMMIT;
Sessao B> SELECT nome FROM cliente WHERE id_cliente = 1; -- nova transacao de B
           'Alterado pela Sessao A'      <- leitura 3
```

Leitura suja não ocorre em nenhum dos dois níveis. Na leitura 1, feita enquanto a Sessão A tinha o `UPDATE` pendente sem `COMMIT`, as duas rodadas devolveram o valor antigo. O MVCC dá a cada transação uma visão consistente formada por versões já confirmadas, mantidas no *undo log*, então o dado não confirmado da Sessão A simplesmente não existe para a Sessão B.

A diferença entre os dois níveis aparece na leitura 2, feita depois do `COMMIT` da Sessão A e ainda dentro da mesma transação da Sessão B. Em **READ COMMITTED**, cada comando abre uma visão nova, então a Sessão B passou a enxergar o valor novo: é a leitura não repetível, duas execuções do mesmo `SELECT` na mesma transação com resultados diferentes. Em **REPEATABLE READ**, a visão é criada uma vez, na primeira leitura da transação, e permanece até o fim, então a Sessão B continuou vendo o valor antigo mesmo com o dado novo já confirmado no banco.

A leitura 3 confirma o mecanismo. Depois do `COMMIT` da Sessão B, a transação seguinte abre uma visão nova e enxerga o valor novo nos dois níveis. Ou seja, o que o **REPEATABLE READ** congela é a visão da transação, não o dado.

O custo da escolha é o esperado: **REPEATABLE READ** entrega consistência de leitura dentro da transação, mas mantém a visão antiga por mais tempo, o que aumenta o *undo log* e, em consultas com intervalo, acrescenta *gap locks* que ampliam a contenção discutida na Seção 4.

7.4 Atualização perdida e o efeito do FOR UPDATE

O experimento 3.1 mostra o bloqueio acontecendo. Este mostra o que acontece quando ele não existe. O caso é o de uma aplicação que lê a quilometragem do veículo, soma o valor rodado e grava de volta, que é o padrão leitura, cálculo, escrita. Duas sessões fazem isso ao mesmo tempo sobre o veículo 2, somando 100 e 50 sobre os 13.400 km da carga.

```
--- Cenário 1: sem controle de concorrência ---
Sessao A> START TRANSACTION; SELECT km_atual INTO @km ... id_veiculo = 2;
           @km = 13400
Sessao B> START TRANSACTION; SELECT km_atual INTO @km ... id_veiculo = 2;
           @km = 13400
Sessao A> UPDATE veiculo SET km_atual = @km + 100 ...; COMMIT;
Sessao B> UPDATE veiculo SET km_atual = @km + 50 ...; COMMIT; (nao bloqueia)
           km_atual final = 13450      esperado pela regra de negocio: 13550

--- Cenário 2: com SELECT ... FOR UPDATE ---
Sessao A> START TRANSACTION; SELECT km_atual INTO @km ... FOR UPDATE;
           @km = 13400
Sessao B> START TRANSACTION; SELECT km_atual INTO @km ... FOR UPDATE;
           (bloqueada ha 3 s)
Sessao A> UPDATE veiculo SET km_atual = @km + 100 ...; COMMIT;
Sessao B> (liberada apos 3.00 s) @km = 13500  <- le o valor ja confirmado por A
Sessao B> UPDATE veiculo SET km_atual = @km + 50 ...; COMMIT;
           km_atual final = 13550      nenhuma atualizacao perdida
```

Sem trava, as duas sessões leem 13.400, cada uma calcula em cima desse valor e a última escrita apaga a anterior: o resultado é 13.450 em vez de 13.550, e os 100 km da Sessão A desapareceram sem nenhum erro. É a atualização perdida, e o ponto importante é que nenhum comando falhou: o banco fez exatamente o que foi pedido, e o dado ficou errado mesmo assim.

Com FOR UPDATE, a leitura da Sessão B fica bloqueada até o COMMIT da Sessão A e, quando volta, lê 13.500, que é o valor já confirmado, e não o 13.400 do início da sua própria transação. Esse detalhe é o que faz a correção funcionar: leitura com bloqueio no InnoDB é leitura corrente, e não leitura do snapshot da transação. O resultado final é 13.550.

Para este caso específico existe uma solução mais simples, que é escrever UPDATE veiculo SET km_atual = km_atual + 100, deixando o cálculo dentro do próprio comando. O FOR UPDATE continua necessário quando a decisão depende do valor lido, que é o caso da seção seguinte.

7.5 Condição de corrida na sp_abrir_locacao

A sp_abrir_locacao, entregue na Parte 2, lê o status do veículo com um SELECT comum, decide a partir dele e só depois insere. Entre a leitura e a escrita existe uma janela. O teste abaixo abre a mesma locação em duas sessões simultâneas, sobre o veículo 5.

```
--- Cenário 3: sp_abrir_locacao (Parte 2), veiculo 5 ---
Sessao A> START TRANSACTION; CALL sp_abrir_locacao(1, 5, 3, 3, @id, @v);
           @id = 18  @v = 569.70 (sem COMMIT)
Sessao B> START TRANSACTION; CALL sp_abrir_locacao(2, 5, 3, 3, @id, @v);
           (bloqueada ha 3 s, no UPDATE do trigger, nao na validacao)
Sessao A> COMMIT;
Sessao B> (liberada) @id = 19  <- a validacao passou
Sessao B> COMMIT;

SELECT ... FROM locacao WHERE id_veiculo = 5 AND data_real_devolucao IS NULL;
  id_locacao | id_cliente | id_veiculo | status
  ---
    18      | 1         | 5         | ABERTA
    19      | 2         | 5         | ABERTA
Duas locacoes abertas para o mesmo veiculo.

--- Cenário 4: sp_abrir_locacao_seguro, veiculo 6 ---
Sessao A> START TRANSACTION; CALL sp_abrir_locacao_seguro(1, 6, 5, 3, @id, @v);
           @id = 18  @v = 1649.70 (sem COMMIT)
Sessao B> START TRANSACTION; CALL sp_abrir_locacao_seguro(2, 6, 5, 3, @id, @v);
           (bloqueada ha 3 s, no SELECT ... FOR UPDATE, antes de qualquer INSERT)
Sessao A> COMMIT;
Sessao B> (liberada) ERROR 1644 (45000): sp_abrir_locacao_seguro: veiculo nao esta disponivel
Sessao B> ROLLBACK;

SELECT ... FROM locacao WHERE id_veiculo = 6 AND data_real_devolucao IS NULL;
  id_locacao | id_cliente | id_veiculo | status
  ---
    18      | 1         | 6         | ABERTA
Uma unica locacao aberta.
```

O resultado é o veículo 5 com duas locações abertas ao mesmo tempo, que é justamente o que a validação deveria impedir. O mecanismo é o seguinte: a leitura de status da Sessão B não toma bloqueio

nenhum e, em **REPEATABLE READ**, enxerga o snapshot em que o veículo ainda está **DISPONIVEL**, então a validação passa. A Sessão B só é bloqueada bem depois, já dentro do **UPDATE** disparado pelo trigger **trg_locacao_ai_status**, quando a decisão errada já foi tomada e a locação já foi inserida. Quando a Sessão A confirma e a Sessão B é liberada, a Sessão B apenas regrava o status que já estava lá e confirma a segunda locação.

A correção está na **sp_abrir_locacao_seguro**, definida no **parte3_concorrencia.sql**. A única diferença de fundo é que a leitura do status passa a ser feita com **FOR UPDATE OF v**, o que move o bloqueio para antes da decisão. Com isso a Sessão B para no **SELECT**, antes de qualquer **INSERT**; quando é liberada, a leitura corrente já mostra o veículo **LOCADO** e a própria validação da procedure dispara o **SIGNAL**. O veículo 6 termina com uma locação aberta apenas.

Duas observações sobre a correção. A primeira é que o **FOR UPDATE** nomeia o alias **v**, e não a tabela **categoria** que participa do mesmo **JOIN**: travar a categoria bloquearia todas as locações da mesma faixa de preço, e não apenas as do mesmo veículo. A segunda é que a procedure segura só funciona se for chamada dentro de uma transação explícita. Em **autocommit**, o bloqueio tomado pelo **SELECT** seria liberado no fim do próprio comando e a janela voltaria a existir.

A **sp_abrir_locacao** original foi mantida no **script_parte2.sql** de propósito, como contraexemplo. O que este experimento mostra é que validação em duas etapas, ler e depois decidir, não é validação sob concorrência, e que o fato de o trigger bloquear em algum momento não salva a operação: o bloqueio precisa acontecer antes da decisão, não depois.

7.6 Outros comportamentos verificados

SERIALIZABLE. Com a Sessão A em **SERIALIZABLE**, um **SELECT** comum dentro de transação passa a tomar bloqueio compartilhado sobre as linhas lidas, e o **UPDATE** da Sessão B, em nível padrão, fica bloqueado até a Sessão A encerrar. É a forma mais forte de isolamento e também a de maior contenção: leitura passa a bloquear escrita.

Atomicidade com triggers. Uma devolução aplicada e depois desfeita com **ROLLBACK** confirma que os efeitos dos gatilhos participam da mesma transação do comando que os disparou. Dentro da transação, o veículo aparecia **DISPONIVEL** com 19.500 km e havia um registro novo em **log_locacao**; para a outra sessão, nada disso existia; depois do **ROLLBACK**, o veículo voltou a **LOCADO** com 19.250 km, a locação voltou a **ABERTA** com devolução nula e o registro de auditoria desapareceu. Ou seja, não existe estado intermediário visível nem efeito de trigger sobrevivendo a um **ROLLBACK**.

8 Observações técnicas

8.1 Índice que sustenta chave estrangeira

Para montar o cenário “antes” da Seção 3 foi preciso derrubar o **idx_veiculo_categoria**. O MySQL recusou:

```
-----
ALTER TABLE veiculo DROP INDEX idx_veiculo_categoria
-----

ERROR 1553 (HY000) at line 1: Cannot drop index 'idx_veiculo_categoria': needed in a foreign key constraint
```

O índice começa por **id_categoria**, que é a coluna da chave estrangeira **fk_veiculo_categoria**, e o InnoDB exige um índice sobre a coluna de referência para validar a integridade a cada escrita. A solução foi derrubar a chave estrangeira antes do índice e recriar as duas depois, o que está no **evidencias_explain.sql**. A conclusão que fica, e que se conecta com a pergunta de reflexão, é que coluna participante de chave estrangeira nunca é “índice a mais” neste modelo: ela é obrigatoriamente indexada, e o custo de escrita correspondente não é opcional.

8.2 Colisão de nomes com a atividade da Aula 14

A atividade de Views e Índices da Aula 14 foi feita no mesmo banco `locadora` e já tinha criado views chamadas `vw_veiculos_disponiveis` e `vw_locacoes_ativas`, com outro conjunto de colunas e outro filtro. Como o roteiro da Parte 2 exige exatamente esses dois nomes, e as views são criadas com `CREATE OR REPLACE`, executar o `script_parte2.sql` sobrescrevia as views da Aula 14 em silêncio, sem nenhum aviso, quebrando as demonstrações daquela atividade.

As views da Aula 14 foram renomeadas para `vw_veiculos_disponiveis_simples` e `vw_locacoes_ativas_join`, e os nomes exigidos pelo roteiro ficaram com as views da Parte 2. As sete convivem no mesmo esquema, como mostra o inventário da Seção 1.1.

8.3 Views com JOIN são atualizáveis no MySQL

Ao documentar as views da Aula 14 partimos da afirmação corrente de que view com JOIN não é atualizável. A medição desmentiu. A `vw_locacoes_ativas_join` aparece com `IS_UPDATABLE = YES` em `information_schema.VIEWS`, e um UPDATE através dela funciona, inclusive alterando coluna de tabela que não é a primeira do FROM. O que o JOIN impede é o DELETE, que retorna `ERROR 1395 (HY000): Can not delete from join view`. O INSERT também falha, mas por outro motivo, que é a existência de colunas NOT NULL sem valor padrão fora da lista de seleção, e retorna `ERROR 1423`. Não atualizável de fato é a view com agregação, como a `vw_faturamento_mensal`, que retorna `ERROR 1288`.

9 Relato da experiência e divisão de tarefas

9.1 Divisão de tarefas

Marcello Azevedo Pinheiro Siqueira. Responsável pela Parte 1, views e índices. Criou as três views pedidas pelo roteiro, escolheu as três consultas frequentes para análise, montou o cenário de medição com `EXPLAIN` antes e depois da criação dos índices e consolidou este relatório. A maior dificuldade foi a Consulta B, em que o índice criado não foi utilizado pelo otimizador. Entender que se tratava de uma decisão de custo, e não de erro na criação do índice, exigiu comparar o campo `filtered` nos dois planos e concluir que a mudança de 11,11% para 70,59% era a prova de que o otimizador havia lido a estatística do índice antes de descartá-lo.

Lucas Basile. Responsável pela Parte 2.1, triggers e auditoria, e pela execução do script de criação do banco. Modelou a tabela `log_locacao`, implementou os três gatilhos e ajustou o gatilho de inserção para não disparar durante a carga histórica do Laboratório 01, que insere locações já finalizadas. A maior dificuldade foi a comparação de `valor_total` no gatilho de auditoria: como o campo é nulo enquanto a locação está aberta, a comparação convencional não detectava a transição de nulo para valor, o que foi resolvido com o operador `<=>`, seguro para nulo.

Miguel Matos. Responsável pela Parte 2.2, procedure e function. Implementou a `sp_abrir_locacao`, com validação de disponibilidade por `SIGNAL SQLSTATE '45000'`, e a `fn_calcula_multa`. A maior dificuldade foi decidir o que gravar em `valor_total` no momento da abertura da locação, resolvida pela consulta ao dicionário de dados entregue na Parte 1, que define o campo como valor efetivamente cobrado e apurado na devolução, e não como valor previsto.

Parte 3, concorrência e transações. Roteiro de execução escrito por Miguel Matos, no arquivo `sql/parte3_concorrenci.sql`, e executado por Miguel Matos e Lucas Basile em duas sessões simultâneas e independentes, conforme a organização em dupla prevista no roteiro. Além dos três experimentos pedidos, o arquivo cobre atualização perdida, a condição de corrida da `sp_abrir_locacao`, `SERIALIZABLE` e atomicidade com triggers.

9.2 Relato da experiência

O ponto do laboratório que mais mudou a forma do grupo entender o assunto foi o Exercício 1.2. A expectativa ao começar era que criar o índice certo resolvesse cada consulta, e a medição mostrou que não é assim: o índice da Consulta B foi criado corretamente, sobre a coluna certa, e mesmo assim o plano continuou sendo varredura completa, porque 70% das linhas caíam dentro do intervalo pedido. A lição prática é que índice é hipótese a ser verificada com **EXPLAIN**, e não solução aplicada por dedução.

Os triggers trouxeram uma dificuldade de natureza diferente. Um gatilho faz um comando produzir efeito em tabela que o comando não menciona, o que é exatamente o objetivo, mas também torna o comportamento menos previsível para quem lê apenas o **INSERT** ou o **UPDATE**. O caso concreto foi o da carga histórica: o primeiro gatilho de inserção marcava como locados veículos que estavam disponíveis, porque disparava também nas locações já finalizadas inseridas pela carga. A solução foi condicionar o disparo ao estado real da locação, e a conclusão foi que todo gatilho precisa ser pensado junto com as cargas em massa, não apenas com a operação normal do sistema.

Na Parte 3, o resultado que mais chamou atenção foi o relatório de deadlock do próprio InnoDB, que descreve o ciclo completo, informa o que cada transação segura e o que espera, e ainda indica qual foi desfeita. Ficou claro que deadlock não é defeito do banco nem erro de programação isolado: é consequência previsível de duas transações pegarem os mesmos recursos em ordens diferentes, e a prevenção é de projeto, por ordem canônica de acesso e transações curtas, não de configuração.

Uma última observação diz respeito ao trabalho em grupo sobre o mesmo banco. A Parte 2 exigia os nomes **vw_locacoes_ativas** e **vw_veiculos_disponiveis**, que já estavam ocupados por views criadas em atividade anterior no mesmo esquema. Como **CREATE OR REPLACE VIEW** substitui sem emitir aviso, a colisão só apareceu quando as demonstrações da atividade anterior pararam de funcionar. A partir disso o grupo passou a conferir o inventário de objetos do esquema antes de rodar qualquer script de criação, o que está registrado na Seção 8.2.